

# **Designed and engineered, artfully — silicon to software.**

BSc Computer Science, Manchester (2023–2026) · Incoming MSc Computer & Embedded Systems Engineering, TU Delft (Sept 2026)

---

|    |                                                                           |            |
|----|---------------------------------------------------------------------------|------------|
| 01 | Quant Trading System (Automaton-QTS)                                      | QUANT      |
| 02 | Cellular-Automaton SoC Accelerator                                        | QUANT      |
| 03 | Aero-Lab — Quadrotor Flight Sim                                           | DRONES     |
| 04 | RRHC vs MPCC: A Reactive Racing Controller That Beats Online Optimisation | MOTORSPORT |
| 05 | Stump — 16-bit RISC CPU closed on Spartan-6 silicon                       | QUANT      |
| 06 | Multicore Computing Labs                                                  | QUANT      |
| 07 | ONI — Local-LLM Agent Harness                                             | OTHER      |
| 08 | DAWgit — Git for Ableton Projects                                         | MUSIC      |
| 09 | FSAI Driverless Racing Simulator                                          | MOTORSPORT |
| 10 | CommonRoad C++ Port                                                       | MOTORSPORT |
| 11 | Slit Light Interference Simulator                                         | OTHER      |
| 12 | CarrLane — Engineering Part Catalog Chatbot                               | OTHER      |
| 13 | AUBRY — Autonomous Cinematography Drone                                   | DRONES     |
| 14 | Proce-Tree — Procedural Tree Generator                                    | OTHER      |
| 15 | FSAI Unity Driving Simulator                                              | MOTORSPORT |

# Quant Trading System (Automaton-QTS)

A multi-signal trading engine, an agent-based market simulator, and a relation-typed contagion-propagation graph that I proved is dead in equities and alive in crypto.

Python PyTorch torchdiffeq NautilusTrader hftbacktest scipy

|                                                        |                                         |                                                                                     |                                                                                                          |
|--------------------------------------------------------|-----------------------------------------|-------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| <div>CODE</div> <div>45.7k LOC</div> <div>Python</div> | <div>TESTS</div> <div>1,314 tests</div> | <div>CRYPTO CONTAGION</div> <div>(RESEARCH-STAGE, N=3)</div> <div>+9.2%/event</div> | <div>TERRA LINKED-PEER</div> <div>SIGNAL (RESEARCH-STAGE,</div> <div>N=3)</div> <div>p=0.0009 @72h</div> |
|--------------------------------------------------------|-----------------------------------------|-------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|

ROLE

Sole author. Built the signal/risk/execution core, the human-gated LLM oversight trail, an agent-based market simulator for synthetic data, and the relation-typed propagation-graph research line (equity negative + crypto positive).

REPO / <https://github.com/RandevRanjit/Automaton-QTS>

# Cellular-Automaton SoC Accelerator

A from-scratch SystemVerilog drawing unit that renders Wolfram cellular automata — programmable rule as a pure lookup, LFSR-seeded, taken through Vivado to silicon and demoed live on a monitor.

SystemVerilog Vivado Artix-7 RISC-V assembly Questa

|                                                     |                                                 |                                            |                                                                               |
|-----------------------------------------------------|-------------------------------------------------|--------------------------------------------|-------------------------------------------------------------------------------|
| <div>ACCELERATOR (BLOCK)</div> <div>4,433 LUT</div> | <div>WHOLE SOC FILL</div> <div>96.0% BRAM</div> | <div>RULE SPACE</div> <div>256 rules</div> | <div>FUNCTIONAL COVERAGE</div> <div>100% stmt/<br/>branch/cond/<br/>FSM</div> |
|-----------------------------------------------------|-------------------------------------------------|--------------------------------------------|-------------------------------------------------------------------------------|

ROLE

Designed unit\_automaton from scratch (programmable rule-as-lookup, 32-bit LFSR seed, 3-state compute/write FSM, byte-lane framestore writes with req/ack backpressure), wrote a coverage-driven testbench to 100% functional coverage, integrated it into the drawing engine replacing a dummy slot, drove it from a RISC-V MMIO program, and closed it through Vivado synthesis + implementation on Artix-7 with a live monitor demo.

# Stump — 16-bit RISC CPU closed on Spartan-6 silicon

A multi-cycle 16-bit RISC processor built bottom-up from RTL primitives in Verilog, then synthesised, placed-and-routed, and timing-closed on a real Spartan-6 at 98% slice occupancy — the first design I carried through the full FPGA back-end flow.

Verilog-2001 Xilinx ISE Spartan-6 (xc6slx4) ModelSim Stump assembly

|                                      |                                       |                                           |                                |
|--------------------------------------|---------------------------------------|-------------------------------------------|--------------------------------|
| SLICE LUTS<br>2,068 / 2,400<br>(86%) | OCCUPIED SLICES<br>591 / 600<br>(98%) | SLICE REGISTERS<br>1,052 / 4,800<br>(21%) | TIMING SCORE<br>0 (timing met) |
|--------------------------------------|---------------------------------------|-------------------------------------------|--------------------------------|

ROLE

Authored the datapath (ALU, barrel shifter, sign-extender, 8-register file with R0=0 / R7=PC, operand muxes), the 3-state control FSM (FETCH/EXECUTE/MEMORY) and the control-decode logic, and implemented the LDST and BCC instruction paths. Top-level board wrapper and 1.5k-line memory model were provided. Wrote test programs in Stump assembly including a Battleship game driving a memory-mapped 8x8 LED matrix.

# Multicore Computing Labs

Three parallelism labs on a 72-core 2-socket NUMA machine — a 132x temporal-blocking Jacobi stencil, a 30.7x NUMA-aware vecadd, and dining philosophers scaling near-linearly to 1.04M meals/s.

C C++20 pthreads OpenMP std::barrier NUMA

|                                                                                                                                     |                                                |                                             |                                                             |
|-------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------|---------------------------------------------|-------------------------------------------------------------|
| TEMPORAL-BLOCKING<br>STENCIL<br><b>132.5× @ 72<br/>cores</b><br><br>SYNC REDUCTION<br>(STENCIL)<br><b>1 barrier /<br/>128 iters</b> | OPENMP STENCIL<br><b>128.6× @ 72<br/>cores</b> | NUMA VECADD SPEEDUP<br><b>30.7× speedup</b> | FINE-SPINLOCK<br>PHILOSOPHERS<br><b>1,039K meals/<br/>s</b> |
|-------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------|---------------------------------------------|-------------------------------------------------------------|

ROLE

Sole author (s94810rr), three labs. Implemented NUMA first-touch pthreads vecadd; four-variant dining philosophers (coarse/fine × mutex/spinlock) with lock-ordering deadlock prevention; and a temporal-blocking 1-D Poisson stencil in both std::thread+std::barrier and OpenMP. Benchmarked on mcore72 (2× Xeon Platinum 8452Y, 72 cores, 2 NUMA nodes).

PRIVATE · CASE STUDY (NO CODE LINK)

FIELD / DRONES

03 / CONTROL

# Aero-Lab — Quadrotor Flight Sim

Hand-rolled 6-DOF quadrotor dynamics plus four racing controllers — PID, MPCC (acados SQP-RTI), a reactive RRHC, and a PPO policy — with PID/MPCC/RRHC benchmarked head-to-head on a 12-course fairness harness.

Python

NumPy

SciPy

acados

CasADi

Gymnasium

|                           |                                     |                                               |                              |
|---------------------------|-------------------------------------|-----------------------------------------------|------------------------------|
| TESTS<br><b>357 tests</b> | CONTROLLERS<br><b>6 controllers</b> | RRHC VS MPCC<br><b>162x cheaper/<br/>tick</b> | CODE<br><b>~8.5k src LOC</b> |
|---------------------------|-------------------------------------|-----------------------------------------------|------------------------------|

ROLE

Sole author. Hand-rolled the 6-DOF rigid-body dynamics (RK4, motor lag, gyroscopic torque, drag), the analytic differential-flatness inner loop shared by every controller, an acados SQP-RTI MPCC, a CTBR PPO env + policy, and a fairness contract (pre-flight + watchdog + post-hoc audit) that enforces identical plant, course, and control rate across all controllers.

PRIVATE REPOSITORY

13 / CONTROL

# AUBRY — Autonomous Cinematography Drone

A camera drone that follows a subject and frames the shot itself — a "follow & react" trailing controller and a "cinematic framing" controller with potential-field obstacle avoidance and a decoupled virtual gimbal, built on the aero-lab racing stack's inner loop.

Python NumPy VisPy

|                                               |                                      |                                                                  |                                                             |
|-----------------------------------------------|--------------------------------------|------------------------------------------------------------------|-------------------------------------------------------------|
| <div>CONTROLLERS</div> <div>2 cinematic</div> | <div>SOURCE</div> <div>470 LOC</div> | <div>OBSTACLE AVOIDANCE</div> <div>APF + 4-pass<br/>detour</div> | <div>INNER LOOP</div> <div>shared, RRHC<br/>untouched</div> |
|-----------------------------------------------|--------------------------------------|------------------------------------------------------------------|-------------------------------------------------------------|

ROLE

Sole author. Designed both cinematic controllers on top of the existing quadrotor stack — a trailing controller that keeps a target arc-length gap behind a moving subject via a rolling breadcrumb spline with iterative obstacle-detour resampling, and a framing controller that holds a relative-pose "front selfie" target with artificial-potential-field repulsion and a decoupled virtual gimbal whose look-at is computed in the runner layer independently of the airframe attitude. Both reuse the shared differential-flatness inner loop without touching RRHC.

PRIVATE REPOSITORY

FIELD / MOTORSPORT

04 / CONTROL

## RRHC vs MPCC: A Reactive Racing Controller That Beats Online Optimisation

Final-year dissertation — a model-free heuristic racing controller, benchmarked against an IPOPT-solved MPCC under an enforced fairness contract.

Python NumPy SciPy CasADi IPOPT Optuna

|                                                     |                                                     |                                                                 |                                                                                 |
|-----------------------------------------------------|-----------------------------------------------------|-----------------------------------------------------------------|---------------------------------------------------------------------------------|
| <div>LAP TIME VS MPCC</div> <div>12.2% faster</div> | <div>PER-STEP COMPUTE</div> <div>153x cheaper</div> | <div>DEADLINE OVERRUNS (ZOH)</div> <div>0 (vs 1,870 MPCC)</div> | <div>CROSS-PHASE SIGNIFICANCE</div> <div>Wilcoxon<br/>W=253,<br/>p=1.9e-5</div> |
|-----------------------------------------------------|-----------------------------------------------------|-----------------------------------------------------------------|---------------------------------------------------------------------------------|

ROLE

Sole author (dissertation). Designed and built the RRHC controller, the MPCC/IPOPT baseline, the 3-DOF Fiala-tyre plant, the fairness-contract harness, and the five-phase statistical evaluation.

PRIVATE REPOSITORY

09 / SYSTEMS • CONTROL

## FSAI Driverless Racing Simulator

Architected and authored the simulation engine (~29k of 34k LOC C++) — RK4 adaptive sub-stepping over three CommonRoad models (7/9/29 state), an OpenGL stereo camera (PBO readback), and ns-budget timing across a software CAN bus. On an 11-person Formula Student AI team I wrote the physics, IO, CAN, and infra; perception by teammates.

C++17 CMake Eigen OpenGL SDL2 ONNXRuntime

|                               |                                                |                                             |                                             |
|-------------------------------|------------------------------------------------|---------------------------------------------|---------------------------------------------|
| CODEBASE<br>~29k LOC (of 34k) | VEHICLE DYNAMICS<br>RK4, 3 models (7-29 state) | STEREO CAMERA<br>PBO readback + ring buffer | REAL-TIME BUDGETS<br>ns clock, 4 subsystems |
|-------------------------------|------------------------------------------------|---------------------------------------------|---------------------------------------------|

ROLE

Architect and sole author of the simulation engine (~29k of 34k LOC). Designed and wrote the velox physics core (RK4 + adaptive sub-stepping, all three CommonRoad vehicle models), the sim loop, the OpenGL FBO stereo camera with PBO readback, the AI-to-VCU CAN interface and UDP S-VCU link, the ns-budget timing + swappable-clock infrastructure, and all common/IO/control libraries. 11-person team project; the ONNX/SIFT/Kalman perception pipeline was teammates' work — it plugs into the camera and pose feeds I built.

REPO / <https://github.com/RandevRanjit/FSAI-Simulation-C>

10 / SYSTEMS • CONTROL

## CommonRoad C++ Port

~10.6k LOC C++20 port of the CommonRoad vehicle models (ST / STD / MB) into a static library, derivative outputs matched bit-for-bit against the Python reference to 1e-13, plus a ~4.5k LOC TypeScript SDK whose JS backend is parity-checked field-by-field against the native one.

C++20 CMake TypeScript YAML SDL2/ImGui

|                              |                                               |                                                      |                                |
|------------------------------|-----------------------------------------------|------------------------------------------------------|--------------------------------|
| CODEBASE<br>~10.6k LOC C++ + | SELECTABLE MODELS<br>ST / STD / MB (29-state) | DERIVATIVE PARITY VS PYTHON<br>ST/STD 1e-13, MB 1e-7 | TEST SUITE<br>19 CTest targets |
|------------------------------|-----------------------------------------------|------------------------------------------------------|--------------------------------|

ROLE

Sole author. Ported the CommonRoad ST/STD/MB vehicle models from the academic Python reference to a C++20 static library ('velox'), built the 'SimulationDaemon' API with a 'ModelTiming' sub-step scheduler, Pacejka tyre model, EV powertrain controller, staged low-speed/loss-of-control safety, and a TypeScript web SDK with a JS backend parity-checked against the native one.

REPO / <https://github.com/RandevRanjit/CommonRoad-CXX-Port>

15 / CONTROL • GRAPHICS

# FSAI Unity Driving Simulator

An earlier Unity/C# Formula Student driving sim — procedurally-generated cone circuits via complex Fourier synthesis with curvature-bounded corners, a dynamic bicycle model (Pacejka tyre + low-speed kinematic blend), driven autonomously by a lookahead path-follower with live telemetry.

C# Unity 6 System.Numerics

|                                                               |                                                                         |                                                              |                                                  |
|---------------------------------------------------------------|-------------------------------------------------------------------------|--------------------------------------------------------------|--------------------------------------------------|
| <div>TYRE MODEL</div> <div><b>Pacejka magic formula</b></div> | <div>TRACK GENERATION</div> <div><b>complex Fourier synthesis</b></div> | <div>LOW-SPEED MODEL</div> <div><b>kinematic blend</b></div> | <div>C# ENGINE</div> <div><b>~2.6k LOC</b></div> |
|---------------------------------------------------------------|-------------------------------------------------------------------------|--------------------------------------------------------------|--------------------------------------------------|

ROLE

Sole author (Formula Student). Wrote the dynamic bicycle vehicle model (Pacejka tyre, aero down/drag-force, low-speed kinematic correction), the procedural cone-track generator (a complex-Fourier path with curvature-derived corner radii and a minimum-radius constraint), the autonomous lookahead path-following controller, and the real-time telemetry + CSV logging — all C# on Unity 6.

PRIVATE REPOSITORY

FIELD / MUSIC

08 / SYSTEMS

# DAWgit — Git for Ableton Projects

A macOS daemon that gives Ableton Live real version control — diffing the music, not the bytes. Semantic .als diff, per-file branching that never touches HEAD, SHA-256 audio LFS, over a Unix-socket IPC bus.

Rust tokio libgit2 (git2-rs) roxmltree flate2 SHA-256



|                                                                                                               |                                                              |                                                                          |                                                                       |
|---------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------|--------------------------------------------------------------------------|-----------------------------------------------------------------------|
| <small>.ALS → TYPED AST</small><br><b>1,306 LOC<br/>parser</b><br><br><small>TESTS</small><br><b>52 tests</b> | <small>SEMANTIC DIFF</small><br><b>21 delta<br/>variants</b> | <small>PER-FILE BRANCH COMMIT</small><br><b>HEAD/index<br/>untouched</b> | <small>AUDIO LFS</small><br><b>SHA-256<br/>content-<br/>addressed</b> |
|---------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------|--------------------------------------------------------------------------|-----------------------------------------------------------------------|

ROLE

Sole author. Built the three-binary system end to end — Rust daemon (FS watcher, async/sync IPC boundary, git object layer, LFS, project discovery, launchd integration), the 1,306-LOC Ableton gzip-XML parser, the semantic differ, the clap CLI, and the SwiftUI menubar app that talks to the daemon over the socket.

PRIVATE REPOSITORY

FIELD / OTHER

07 / SYSTEMS

# ONI — Local-LLM Agent Harness

79k-LOC single-crate Rust harness driving a local llama.cpp / MLX server through a 33-tool fenced-code protocol, a graph working-memory, and harness-managed compaction — with 2,151 tests and µs/ns Criterion latency contracts.

Rust tokio request criterion llama.cpp MLX

|                                                                                                                        |                                                                                                |                                                   |                                                             |
|------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|---------------------------------------------------|-------------------------------------------------------------|
| <small>CODEBASE</small><br><b>79.2k LOC<br/>Rust</b><br><br><small>LOOP-DETECTOR BUDGET</small><br><b>&lt;50 ns/KB</b> | <small>TESTS</small><br><b>2,151 tests</b><br><br><small>BACKENDS</small><br><b>3 backends</b> | <small>PARSER BUDGET</small><br><b>&lt;100 µs</b> | <small>TOOL DISPATCH BUDGET</small><br><b>&lt;5 ms/call</b> |
|------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|---------------------------------------------------|-------------------------------------------------------------|

ROLE

Sole author. Designed and built the whole harness: the OMTF v0.4 fenced-code tool protocol and its 33-tool parser, the backend seam abstracting three inference servers, the graph working-memory with BFS gather and harness-managed compaction, the LIT/SIT stream-loop detectors, and the process-group SIGKILL sandbox. Every behaviour-changing feature is greenlit, rollback-tracked, and validated by scored A/B runs.

PRIVATE REPOSITORY

11 / GRAPHICS

# Slit Light Interference Simulator

A 3D single-slit Fraunhofer diffraction lab — the  $\text{sinc}^2$  intensity envelope solved per-fragment on the GPU, wrapped in a Three.js scene with draggable apparatus, live wavelength/slit/distance controls and a real-time measurement readout.

JavaScript Three.js GLSL Tweakpane Webpack

|                                                                                                       |                                                                           |                                                |                                                                              |
|-------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------|------------------------------------------------|------------------------------------------------------------------------------|
| <div>DIFFRACTION MODEL</div> <div><b>sinc<sup>2</sup></b><br/><b>Fraunhofer,</b><br/><b>GPU</b></div> | <div>SHADER PROGRAMS</div> <div><b>3 shader</b><br/><b>programs</b></div> | <div>ENGINE</div> <div><b>~1.65k LOC</b></div> | <div>WAVELENGTH→COLOUR</div> <div><b>380–780 nm</b><br/><b>CIE→RGB</b></div> |
|-------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------|------------------------------------------------|------------------------------------------------------------------------------|

ROLE

Sole author. Wrote the interference vertex/fragment shader pair ( $\text{sinc}^2$  envelope, SI-unit parametrisation), the wavelength→RGB conversion, the full Three.js scene graph, the draggable transform-controlled apparatus, the live measurement panel, and the particle-haze system with its sorted exponential-search culling.

REPO / <https://github.com/RandevRanjit/Slit-Light-Interference-Sim>

12 / SYSTEMS

# CarrLane — Engineering Part Catalog Chatbot

Three-tier function-calling assistant that turns natural-language part queries into typed API calls over a scraped engineering catalog — built during an industry internship.

Python FastAPI SQLAlchemy Node/Express React Chakra UI

|                                                                                                                                |                                                            |                                                                                       |                                                                                     |
|--------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------|---------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| <div>CATALOG (PROTOTYPE)</div> <div><b>314 parts</b></div> <div>DATES</div> <div><b>Jun–Aug 2025,</b><br/><b>Chennai</b></div> | <div>COMPANY CATALOG</div> <div><b>10,000+ parts</b></div> | <div>LLM TOOLS</div> <div><b>9 typed</b><br/><b>function</b><br/><b>schemas</b></div> | <div>API TESTS</div> <div><b>8 async</b><br/><b>endpoint</b><br/><b>tests</b></div> |
|--------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------|---------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|

ROLE

AI / Software Engineering Intern. Built the catalog ingestion, the typed REST API over it, and the function-calling orchestration layer that maps natural-language queries to catalog lookups.

PRIVATE REPOSITORY

14 / GRAPHICS

# Proce-Tree — Procedural Tree Generator

A real-time procedural tree grown from a resource-flow model — branches that steer away from their own crowded canopy, cross-sectional area conserved at every fork, depth-decaying splits, and camera-facing instanced leaves, in Three.js.

JavaScript

Three.js

Vite

|                                                                           |                                                                                              |                                                                     |                                              |
|---------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|---------------------------------------------------------------------|----------------------------------------------|
| <div>GROWTH MODEL</div> <div><b>resource-flow</b><br/><b>+ area</b></div> | <div>BRANCH STEERING</div> <div><b>canopy-</b><br/><b>density</b><br/><b>avoidance</b></div> | <div>LEAVES</div> <div><b>instanced</b><br/><b>billboards</b></div> | <div>ENGINE</div> <div><b>~520 LOC</b></div> |
|---------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|---------------------------------------------------------------------|----------------------------------------------|

## ROLE

Sole author. Designed the growth algorithm — a binary-splitting branch model fed by a "growth resource" that is consumed in proportion to cross-sectional area and conserved across each fork, with branches that compute a weighted-average canopy position and grow *away* from it — plus the Three.js scene, the per-frame branch geometry, and the camera-facing instanced leaf billboards.

REPO / <https://github.com/RandevRanjit/Proce-Tree>